

Índice

1. Descripción del Taller	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Marco Teórico	2
3.1. Redes neuronales	2
3.2. Funciones de activación	2
3.3. Paralelismo y GPU	3
4. Metodología	3
4.1. Configuración del entorno	3
4.2. Dataset	3
4.3. Arquitectura del modelo	3
4.4. Entrenamiento	3
4.5. Extracción de parámetros	3
4.6. Inferencia en CPU	3
4.7. Implementación en GPU	3
5. Resultados	3
5.1. Comparación de tiempos	4
6. Análisis de resultados	4
7. Conclusiones	4
8. Repositorio o Evidencia de Ejecución	5

1. Descripción del Taller

El presente taller tiene como objetivo implementar una red neuronal artificial para la clasificación de dígitos escritos a mano, utilizando el conjunto de datos MNIST. Posteriormente, se busca reconstruir el proceso de inferencia de dicha red mediante el uso de CUDA en GPU, a partir de los parámetros previamente entrenados.

El enfoque principal consiste en comprender el funcionamiento interno de una red neuronal, extrayendo sus pesos y sesgos, y replicando las operaciones matemáticas que realiza cada capa sin depender directamente de librerías de alto nivel como Keras.

2. Objetivos

2.1. Objetivo general

Implementar y analizar una red neuronal para clasificación de imágenes, reconstruyendo su inferencia mediante CUDA en GPU.

2.2. Objetivos específicos

- Entrenar una red neuronal utilizando el conjunto de datos MNIST.
- Extraer los pesos y bias de las capas del modelo entrenado.
- Implementar la inferencia manual de la red en CPU.
- Desarrollar kernels CUDA para ejecutar la inferencia en GPU.
- Comparar el rendimiento entre CPU y GPU.

3. Marco Teórico

3.1. Redes neuronales

Una red neuronal artificial está compuesta por múltiples neuronas organizadas en capas. Cada neurona recibe entradas, las pondera mediante pesos y genera una salida.

Matemáticamente, una neurona se describe como:

$$z = \sum_{i=1}^n x_i w_i + b$$

donde x_i son las entradas, w_i los pesos y b el sesgo.

3.2. Funciones de activación

Para introducir no linealidad en el modelo se utiliza la función ReLU:

$$f(z) = \max(0, z)$$

En la capa de salida se emplea la función Softmax para convertir los resultados en probabilidades:

$$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

3.3. Paralelismo y GPU

Las GPU permiten ejecutar múltiples operaciones de manera simultánea. En el contexto de redes neuronales, cada neurona puede calcularse de forma independiente, lo que hace posible aprovechar el paralelismo.

CUDA es una tecnología que permite ejecutar código en GPU mediante la definición de kernels, donde cada hilo realiza una operación específica.

4. Metodología

4.1. Configuración del entorno

El desarrollo se realizó en Google Colab utilizando Python. Se emplearon librerías como TensorFlow, NumPy, Matplotlib, OpenCV y Numba para CUDA.

4.2. Dataset

Se utilizó el conjunto MNIST, compuesto por imágenes de 28x28 píxeles. Estas se normalizaron dividiendo entre 255 para obtener valores entre 0 y 1.

4.3. Arquitectura del modelo

La red neuronal implementada consta de:

- Una capa de entrada de 784 neuronas.
- Cuatro capas ocultas de 128 neuronas con activación ReLU.
- Una capa de salida de 10 neuronas con activación Softmax.

4.4. Entrenamiento

El modelo se entrenó durante 10 épocas utilizando el optimizador Adam y la función de pérdida de entropía cruzada.

4.5. Extracción de parámetros

Se extrajeron los pesos y bias de las capas densas del modelo entrenado para su posterior uso en la reconstrucción manual de la red.

4.6. Inferencia en CPU

Se implementó manualmente la propagación hacia adelante mediante operaciones de álgebra lineal utilizando NumPy.

4.7. Implementación en GPU

Se desarrollaron kernels CUDA utilizando Numba, donde cada hilo calcula la salida de una neurona, permitiendo paralelizar el proceso.

5. Resultados

El modelo entrenado alcanzó una precisión aproximada del 97 % sobre el conjunto de prueba.

La inferencia implementada en CUDA logró clasificar correctamente imágenes del dataset MNIST, así como una imagen externa dibujada manualmente.

5.1. Comparación de tiempos

Se obtuvo el siguiente comportamiento:

- Tiempo CPU: bajo
- Tiempo GPU: mayor que CPU

Esto indica que, para este caso específico, la GPU no resultó más eficiente.

6. Análisis de resultados

El modelo entrenado logró una alta precisión en la clasificación de dígitos, lo cual indica que la arquitectura seleccionada fue adecuada para el problema planteado. La reconstrucción de la inferencia mediante CUDA permitió replicar correctamente el comportamiento de la red neuronal, obteniendo predicciones coherentes tanto en el conjunto de prueba como en imágenes externas.

Desde una perspectiva conceptual, el proceso puede entenderse mediante una analogía: la red neuronal funciona como una cadena de filtros o decisiones sucesivas. Cada capa recibe información, la transforma y la transmite a la siguiente, refinando progresivamente la interpretación de la imagen. Es similar a observar una imagen borrosa y, paso a paso, ir identificando patrones más claros hasta reconocer finalmente el dígito representado.

En la implementación en CPU, todas estas operaciones se ejecutan de manera secuencial, es decir, una neurona tras otra. En cambio, en la GPU se intenta que múltiples neuronas trabajen simultáneamente, como si varios observadores analizaran diferentes partes del problema al mismo tiempo.

Sin embargo, en este caso particular, la GPU no mostró una mejora en el tiempo de ejecución. Esto puede explicarse considerando que el problema es de tamaño reducido: la red neuronal utilizada es relativamente pequeña y el procesamiento se realiza sobre una imagen a la vez. En consecuencia, la cantidad de trabajo no es suficiente para aprovechar completamente el paralelismo de la GPU.

Adicionalmente, el proceso de transferencia de datos entre la memoria del CPU y la GPU introduce un costo adicional. Este costo puede superar el beneficio del paralelismo cuando el volumen de datos es bajo, lo que explica por qué el tiempo total en GPU resultó mayor.

En términos generales, los resultados obtenidos confirman que la implementación en CUDA es funcional y conceptualmente correcta, pero también evidencian que el uso de GPU debe evaluarse según el tamaño del problema y la cantidad de datos a procesar.

7. Conclusiones

- Se logró entrenar una red neuronal para clasificación de dígitos.
- Se extrajeron correctamente los parámetros del modelo.
- Se reconstruyó la inferencia de la red en CPU y GPU.
- La implementación en CUDA funcionó correctamente.
- La GPU no siempre es más rápida en problemas pequeños.
- Se comprendió la relación entre redes neuronales y paralelismo.

8. Repositorio o Evidencia de Ejecución

https://colab.research.google.com/drive/1K66oVebHmgbiWXjSDkyGVdVz_d4t3a_h?usp=sharing